

Student Grade Management System

Project Report

Course: Programming in Java
Submitted by: [Student Name]
Registration Number: [Registration Number]
Section: [Section]
Date of Submission: [Submission Date]

A command-line Java application demonstrating Object-Oriented Programming, the Collections Framework, exception handling, file I/O, and multithreading.

1. Introduction

This report documents the design, implementation, and testing of the **Student Grade Management System**, a command-line Java application built for the Programming in Java course. The application allows an operator to add student records, view and search them, delete records, calculate results and grades, sort students by performance, generate class-wide statistics, and persist all data to a file so it survives across program restarts. The system also performs automatic background saving using a daemon thread.

The project was chosen to demonstrate a broad range of core Java concepts taught in the course, including object-oriented design, the Collections Framework, custom exception handling, file-based I/O, and basic concurrency using threads and synchronization.

2. Problem Statement

Manual management of student marks can make searching, calculating results, assigning grades, and maintaining records time-consuming and error-prone. A simple Java-based system is required to store student information, validate marks, calculate results consistently, and preserve records between executions.

3. Objectives

- Provide a simple command-line interface for managing student academic records.
- Allow students to be added, viewed, searched, and deleted reliably.
- Calculate total marks, percentage, and grade automatically for each student.
- Validate marks so that invalid input does not corrupt student records.
- Prevent duplicate student IDs from being added to the system.
- Sort students by percentage and generate class-wide statistics (average, highest, lowest).
- Persist student data to a file so records are retained across application restarts.
- Perform automatic background saving so data is not lost if the application closes unexpectedly.

4. Functional Requirements

The system implements the following functional modules:

Module 1 — Student Management

- Add student
- View all students
- Delete student
- Prevent duplicate student IDs

Module 2 — Result Management

- Store subject marks (Java, Mathematics, Digital Logic)
- Calculate total marks
- Calculate percentage
- Assign grade
- Display student result

Module 3 — Search

- Search a student using their student ID

Module 4 — Sorting and Analytics

- Sort students by percentage (descending)
- Calculate class average
- Find highest percentage
- Find lowest percentage

Module 5 — File Persistence

- Save all students to a text file
- Load students automatically when the application starts

Module 6 — Automatic Saving

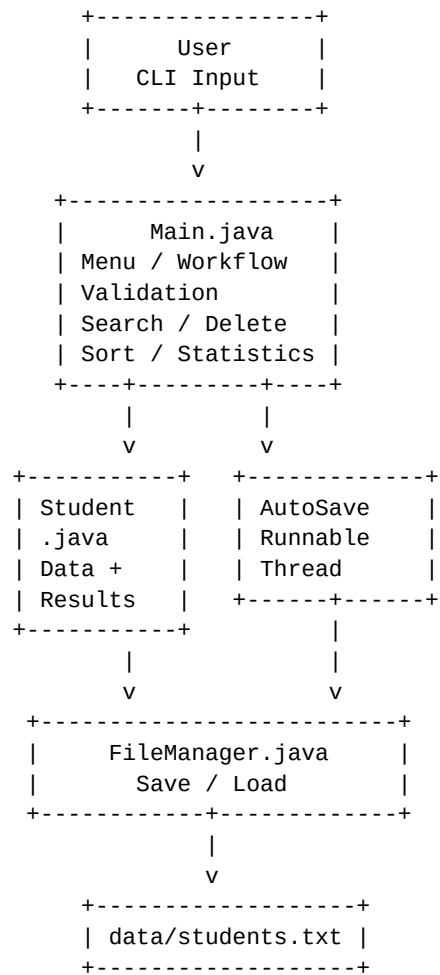
- Background auto-save every 30 seconds
- Implemented using Runnable and Thread
- Uses synchronization to protect shared data

5. Non-Functional Requirements

Requirement	Description
Usability	The system uses a simple numbered command-line menu that is easy to navigate.
Reliability	Student data is persisted to a file so records survive application restarts.
Maintainability	The system is separated into multiple Java classes, each with a distinct responsibility.
Error Handling	Invalid marks are rejected using a custom InvalidMarksException; file I/O errors are also handled.
Resource Efficiency	An ArrayList holds records in memory; file operations occur only when required or during periodic auto-s
Concurrency Safety	The shared student collection is accessed using synchronized blocks to prevent unsafe concurrent acce

6. System Architecture

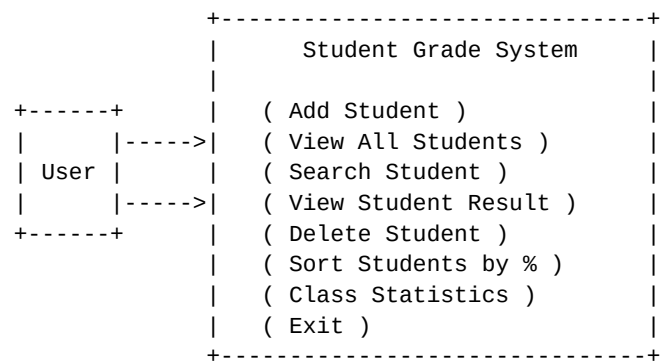
The diagram below shows the high-level architecture of the application.



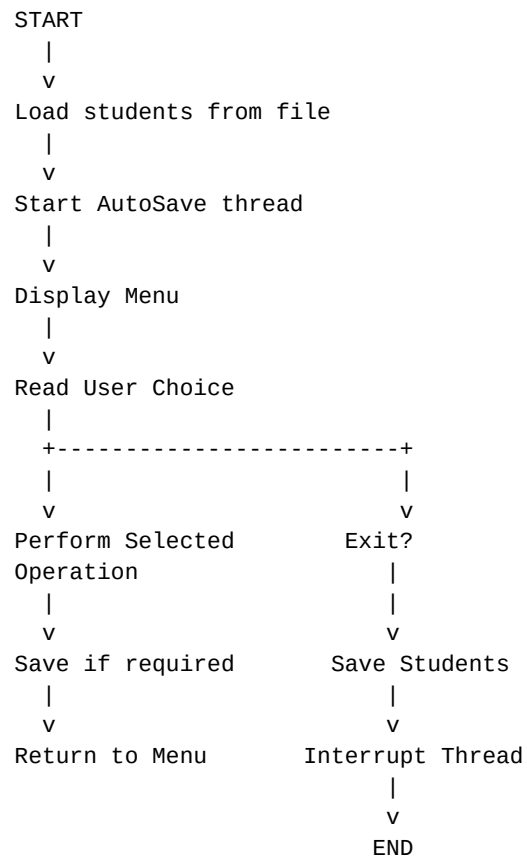
7. Use Case Diagram

Actor: User

Use Cases: the User interacts with all of the following use cases.



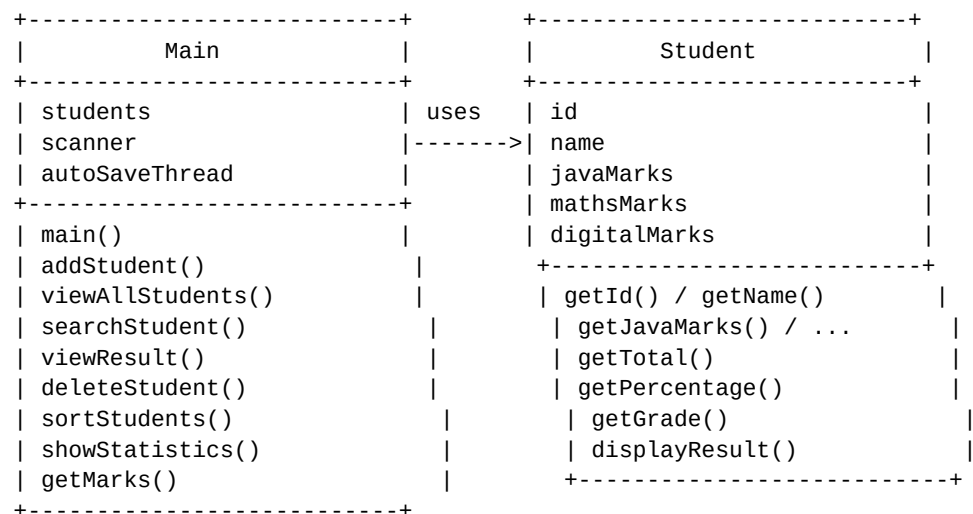
8. Workflow / Process Flow Diagram



9. Sequence Diagram (Add Student)

```
User
|
| Enter student details
v
Main
|
| Validate ID and marks
|
| Create Student object
v
Student
|
| Return object
v
Main
|
| saveStudents()
v
FileManager
|
| write record
v
students.txt
|
| save complete
v
FileManager
|
v
Main
|
| Success message
v
User
```

10. Class Diagram



Main --> FileManager

```
+-----+
|   FileManager   |
+-----+
| FILE_NAME       |
+-----+
| saveStudents()  |
| loadStudents()  |
+-----+
```

Main --> AutoSave

```
+-----+
|   AutoSave      |
| implements Runnable |
+-----+
| students         |
+-----+
| run()            |
+-----+
```

Main --> InvalidMarksException

```
+-----+
| InvalidMarksException |
| extends Exception     |
+-----+
| constructor(message)  |
+-----+
```

11. Storage Design

The project uses flat-file persistence rather than a relational database, so an Entity-Relationship (ER) diagram is **not applicable**. Student records are stored as comma-separated lines in a plain text file:

```
data/students.txt
```

Record format:

```
ID, Name, JavaMarks, MathsMarks, DigitalMarks
```

Example records:

```
13, rahul, 85.0, 92.0, 90.0  
34, kirmada, 75.0, 78.0, 78.0
```

Field	Type	Meaning
ID	int	Student ID (unique)
Name	String	Student name
JavaMarks	double	Java marks (0-100)
MathsMarks	double	Mathematics marks (0-100)
DigitalMarks	double	Digital Logic marks (0-100)

12. Design Decisions and Rationale

ArrayList

Used because the number of students can grow dynamically during program execution.

Iterator

Used during deletion to safely remove an element while iterating over the ArrayList.

Comparator

Used to sort students by their calculated percentage without modifying the Student class's core structure.

Custom Exception

InvalidMarksException gives a clear, domain-specific validation error when marks fall outside the 0-100 range.

File I/O

BufferedReader and BufferedWriter are used to persist student records efficiently to a text file.

Runnable and Thread

AutoSave implements Runnable and executes in a background daemon thread, saving data periodically without blocking the main menu.

Synchronization

The shared student ArrayList is accessed by both the main thread and the AutoSave thread, so synchronized blocks are used to prevent unsafe concurrent access.

13. Implementation Details

Main.java

Responsible for program startup, loading students, starting AutoSave, displaying the menu, and handling all menu operations: adding, searching, viewing results, deleting, sorting, computing statistics, and exiting.

Student.java

Responsible for holding student data (encapsulated fields), and calculating total marks, percentage, grade, and displaying a formatted result.

FileManager.java

Responsible for creating the data directory, saving records to students.txt, loading records at startup, and handling file I/O exceptions.

AutoSave.java

Responsible for implementing Runnable, sleeping for 30 seconds between cycles, saving data automatically, and stopping cleanly when interrupted.

InvalidMarksException.java

Represents invalid marks input (outside 0-100) as a custom checked exception, forcing calling code to handle invalid input explicitly.

Grading Logic

Percentage is calculated as:

$$(\text{Java Marks} + \text{Mathematics Marks} + \text{Digital Logic Marks}) / 3$$

Total is calculated as:

$$\text{Java} + \text{Mathematics} + \text{Digital Logic} \quad (\text{maximum } 300)$$

Percentage Range	Grade
90 and above	A+
80 to 89.99	A
70 to 79.99	B
60 to 69.99	C
50 to 59.99	D
Below 50	F

14. Screenshots / Results

No screenshots are attached to this report. The results below are actual console output captured during manual command-line testing of the application.

Menu Display

```
=====
      STUDENT GRADE MANAGEMENT
=====
1. Add Student
2. View All Students
3. Search Student
4. View Student Result
5. Delete Student
6. Sort Students by Percentage
7. Class Statistics
8. Exit
=====
```

Viewing All Students

```
ID: 34 | Name: kirmada | Percentage: 77.00% | Grade: B
ID: 13 | Name: rahul | Percentage: 89.00% | Grade: A
```

Class Statistics

```
===== CLASS STATISTICS =====
Total Students : 3
Class Average  : 69.00%
Highest Score  : 89.00% (rahul)
Lowest Score   : 41.00% (moodi)
```

Duplicate ID Validation

```
Enter choice: 1
Enter student name: dede
Enter student ID: 45
Error: Student ID already exists.
```

AutoSave Output

```
[AutoSave] Student data saved automatically.
```

15. Testing Approach

The application was tested manually through the command-line interface. Each functional module was exercised with valid and invalid input to confirm correct behaviour.

#	Test Case	Result
1	Add Student	Pass
2	Duplicate ID rejected	Pass
3	Class Statistics calculation	Pass

4	View All Students	Pass
5	Persistence across restart	Pass
6	AutoSave (background thread)	Pass
7	Invalid marks (outside 0-100) rejected	Pass
8	Search (found and not found cases)	Pass
9	Delete Student	Pass
10	Sort Students by Percentage	Pass

16. Challenges Faced

- 1 Maintaining persistent records correctly between application executions.
- 2 Debugging a record-loading problem that appeared after restarting the application.
- 3 Preventing duplicate student IDs from being added.
- 4 Validating marks to ensure they stayed within the 0-100 range.
- 5 Handling file I/O exceptions gracefully.
- 6 Coordinating the AutoSave thread with the shared student ArrayList.
- 7 Using synchronization correctly to avoid concurrency issues.
- 8 Keeping multiple classes organized with clear responsibilities.

Persistence Debugging Example: During testing, the application correctly showed 3 students in a running session, but only 2 records were loaded after restarting the program. Investigation traced the issue to the record-parsing logic in `FileManager.java`. The fix was to split each line into a maximum of 5 parts and validate the resulting array length before parsing:

```
String[] data = line.split(",", 5);

if (data.length != 5) {
    // skip invalid record
}
```

After applying this fix, all records were correctly loaded and persisted across restarts, and this was confirmed through repeated manual testing.

17. Learnings and Key Takeaways

This project reinforced several core Java concepts in a practical setting, including:

- Object-oriented programming and encapsulation
- Class relationships and modular design
- The Collections Framework: ArrayList, Iterator, and Comparator
- Custom exception handling
- File input/output with BufferedReader and BufferedWriter
- Threads, Runnable, and daemon threads
- Synchronization for safe concurrent access to shared data
- Practical debugging of a real persistence issue
- Manual testing methodology
- Organizing and submitting a project using Git/GitHub

18. Future Enhancements

The current implementation does not include a database, GUI, authentication, or web interface. These are identified as possible future enhancements:

- JDBC database integration
- MySQL/SQLite database
- GUI using JavaFX/Swing
- Web interface
- Authentication and role-based access
- Support for more subjects
- Semester-wise results
- Attendance management
- PDF report generation
- Automated unit tests
- Backup and restore
- Search by name and filtering
- More advanced statistics
- Logging

19. References

- Oracle Java SE Documentation — docs.oracle.com/javase
- Java Collections Framework Documentation
- Course materials: Programming in Java

20. Conclusion

The Student Grade Management System successfully meets its stated objectives: it allows student records to be added, viewed, searched, deleted, sorted, and analyzed through a simple command-line interface, while persisting all data to a file with automatic background saving. The project demonstrates a practical combination of object-oriented design, the Collections Framework, custom exception handling, file I/O, and multithreading with synchronization — all core concepts from the Programming in Java course. Manual testing confirmed that every functional module, including edge cases such as duplicate IDs, invalid marks, and data persistence across restarts, behaves correctly.